

مواصفات التصميم الفني لنظام إدارة سجلات الطلاب

(Student Records Management System)

١. نظرة عامة على بنية النظام (System Architecture Overview)

يعتمد نظام إدارة سجلات الطلاب على بنية برمجية مكونة من طبقات (Layered Architecture)، وهو خيار استراتيجي يهدف إلى تعزيز قابلية الصيانة (Maintainability) وفصل الاهتمامات (Separation of Concerns) من منظور هندسي، يضمن هذا التقسيم استقلالية كل مكون، مما يسمح بتطوير واجهة المستخدم أو تعديل منطق الأعمال دون المساس بهيكل البيانات الأساسي. تم اختيار نمط "الملفات الرأسية فقط" (Header-only files) لتبسيط عملية النشر والتوزيع وتجنب تعقيدات ربط الملفات (Linking)؛ ومع ذلك، كمُعَمَّرين للأنظمة، ندرك أن هذا التصميم يوفر سرعة في التطوير الأولي ولكنه قد يزيد من زمن التجميع (Compilation Time) في المشروعات الضخمة، وهو مقايضة مقبولة في سياق هذا النظام لضمان التماسك والسهولة.

التحليل الفني للطبقات الأربع:

- **طبقة واجهة المستخدم (UI Layer):** وتتمثل في `clsMainScreen` و `clsStudentUI` وظيفتها الأساسية هي إدارة التفاعل مع المستخدم النهائي وتنسيق المخرجات، بعيداً عن منطق التحقق أو التخزين.
- **طبقة منطق الأعمال (Business Logic Layer):** ويمثلها `clsStudentManager`. تُعد "المحرك" (Engine) للنظام، حيث تدير العمليات الجوهرية (CRUD) والفرز والإحصائيات، وتعمل كوسيط ذكي بين الواجهة والبيانات.
- **طبقة البيانات (Data/Model Layer):** وتتمثل في `clsStudent`. تُعنى بتعريف كيان الطالب وتغليف بياناته وحمايتها، مما يضمن سلامة تمثيل السجل البرمجي.
- **طبقة الأدوات المساعدة (Utilities Layer):** وتشمل `clsInputValidate` و `clsUI_Utility` توفر وظائف دعم عرضية مثل التحقق الصارم من المدخلات وتنسيق الجداول، مما يعزز من نظافة الكود (Clean Code).

مخطط تدفق العلاقات الفنية:

يبدأ تدفق التنفيذ من `main.cpp` الذي يتولى تحميل البيانات الأولية، ثم ينتقل التحكم إلى `clsMainScreen` لإدارة الحلقة الرئيسية. عند طلب عملية معينة، تستدعي الشاشة الرئيسية الوظائف المقابلة في `clsStudentUI`. تقوم هذه الأخيرة بجمع البيانات والتحقق من صحتها عبر `clsInputValidate` الذي يستفيد من أدوات `clsUI_Utility` وبمجرد اكتمال البيانات، يتم تمريرها إلى `clsStudentManager` الذي يعالجها ككائنات من نوع `clsStudent` ويقوم بتحديث الذاكرة والملفات الدائمة.

هذا التنسيق الطبقي يمهد الطريق لفهم دقيق لنمذجة البيانات التي تشكل العمود الفقري لهذا الهيكل.

main()

|__ clsStudentManager::LoadDataFromBinary()

| |__ Import_From_Binary_File() يقرأ Students.data ويعبئ v_Students()

|__ clsMainScreen::ShowMainMenu()

|__ while (!Exit)

|__ Show List

|__ clsInputValidate::Verify_Integers()

|__ switch (choice)

|__ clsStudentUI تقوم باستدعاء الدالة المناسبة من

٢. نمذجة البيانات وهيكـل الكيان (Data Modeling: clsStudent)

يمثل كائن الطالب (clsStudent) حجر الزاوية المعماري للنظام. تم تصميم هذا الكيان باستخدام مبدأ "التغليف" (Encapsulation) الصارم، حيث تُحفظ الخصائص كمحددات وصول خاصة (Private) مع توفير واجهات وصول (Getters/Setters) لضمان عدم العبث بسلامة البيانات.

تحليل الخصائص البرمجية وعناصر الكيان:

الغرض الفني	نوع البيانات	الحقل البرمجي
المعرف الأساسي (Primary Key) ويبدأ من ١٠٠٠.	int	_RollNumber
الاسم الأول للطالب.	char[40]	_FirstName
اسم العائلة.	char[40]	_LastName
القسم) يقتصر على: CS, IT, ME, EE, CE) :	char[40]	_Department
المستوى الدراسي (نطاق محدد من ١-٨).	int	_Semester
المعدل التراكمي (نطاق دقيق ٠,٠٠ - ٤,٠٠).	float	_cGPA
تاريخ الميلاد بتنسيق (dd/mm/yyyy)	char[12]	_DateofBirth
رقم الهاتف (محدد بـ ٩ أرقام).	char[20]	_PhoneNumber
البريد الإلكتروني الرسمي.	char[40]	_Email
حالة السجل (نشط أو غير نشط) للإدارة المنطقية.	bool	_IsActive

تقييم القرارات التقنية (كفاءة التخزين الثنائي):

لقد اتخذنا قراراً معمارياً باستخدام مصفوفات الأحرف ذات الحجم الثابت (char[]) بدلاً من std::string. هذا القرار حيوي لضمان "استقرار حجم الكائن في الذاكرة". بما أن std::string تعتمد على التخصيص الديناميكي للذاكرة (Heap)، فإن محاولة حفظ الكائن مباشرة ستؤدي إلى حفظ مؤشرات غير صالحة. استخدام char[] يضمن أن حجم الكائن ثابت ومعروف مسبقاً، مما يسمح لنا باستخدام reinterpret_cast<char*> مع sizeof(clsStudent) للقراءة والكتابة المباشرة وبسرعة فائقة من الملفات الثنائية.

هذا التعريف الدقيق للبيانات هو ما يمنح طبقة منطق الأعمال القدرة على تنفيذ العمليات المعقدة بكفاءة.

٣. طبقة منطق الأعمال وإدارة العمليات (Business Logic: clsStudentManager)

تعمل clsStudentManager كمحرك مركزي (The Engine) للنظام. تم استخدام ميزة inline static المتوفرة في معيار C++17 لتعريف المنتج العالمي للطلاب (_vStudents) داخل الملف الرئيسي مباشرة، مما يضمن وجود نسخة واحدة موحدة من البيانات في الذاكرة دون الوقوع في أخطاء التحديد المتعدد. (Multiple Definition)

تفصيل العمليات الأساسية: (CRUD Logic)

١. توليد المعرفات: تستخدم الدالة Get_Unique_Roll منطقاً يعتمد على جلب أعلى رقم جلوس موجود في النظام وإضافة (+١) إليه، مع البدء من القيمة الافتراضية ١٠٠٠.

٢. كفاءة الوصول: (Pointer Logic) تم تصميم الدالة FindByRollNumberPtr لتعيد مؤشراً (Pointer) للكائن الأصلي داخل المنتج. هذا الاختيار يتفوق معمارياً على إعادة نسخة من الكائن؛ لأنه يمنع عبء النسخ (Copy Overhead) ويسمح بإجراء التعديلات مباشرة في الذاكرة لتعكس فوراً على المنتج الساكن.

٣. إدارة الحذف: يدعم النظام الحذف المنطقي (Logical Delete) عبر تغيير حالة _IsActive لتسريع العمليات، أو الحذف الفعلي (Actual Delete) باستخدام المكررات (Iterators) لمسح السجل نهائياً.

التحليل التحليلي للإحصائيات والفرز:

يتم جمع البيانات عبر هيكل stStatistics باستخدام "تمريرة واحدة (Single Pass)" على المتجه، مما يحقق تعقيداً زمنياً مثالياً قدره $O(n)$ ، أما في عمليات الترتيب (Sorting)، فقد تم دمج دوال "Lambda" مع خوارزمية `std::sort` القياسية؛ هذا يوفر مرونة عالية تتيح للمطورين تغيير معايير الترتيب (تصاعدي/تنازلي حسب المعدل أو الاسم) بأقل قدر من الكود وبأقصى أداء ممكن.

هذه العمليات تضمن استقرار البيانات في الذاكرة، وهو ما يتم تأمينه لاحقاً عبر آليات التخزين الدائم.

٤. آليات استمرارية البيانات (Data Persistence Mechanisms)

تعتبر استمرارية البيانات (Data Persistence) الركيزة التي تحول التطبيق من أداة مؤقتة إلى نظام مهني. يطبق النظام استراتيجية مزدوجة تضمن الأمان التقني وسهولة التبادل.

المقارنة التقنية لأساليب التخزين:

المعيار	التخزين الثنائي (Students.data)	التصدير النصي (StudentsData.txt)
التقنية	<code>reinterpret_cast</code> مع <code>sizeof</code>	stringstream مع محددات (Delimiters)
السرعة	فائقة (إزاحة مباشرة للذاكرة)	أبطأ (تحويل البيانات إلى نصوص)
كفاءة المساحة	مثالية (تخزين ثنائي مضغوط)	أقل كفاءة (تعتمد على طول المحتوى)
القراءة البشرية	غير مقروء (للحماية والكفاءة)	مقروء وسهل الفتح عبر Excel/CSV

بروتوكول النسخ الاحتياطي: (Backup & Restore)

يتبع النظام بروتوكولاً صارماً للنسخ الاحتياطي؛ حيث تقوم عملية Backup بأخذ لقطة (Snapshot) من حالة الذاكرة الحالية وحفظها في الملف الثنائي Students.data. أما عملية Restore فهي تعتمد على مسح المتجه الحالي وإعادة تحميله من الملف، مما يضمن سلامة البيانات عند حدوث أي خطأ في الجلسة الحالية.

٥. بروتوكولات التحقق واجهة المستخدم (Validation & UI Utilities)

تمثل طبقة التحقق خط الدفاع الأول (First Line of Defense) لضمان تكامل البيانات (Data Integrity) ومنع حالات الانهيار غير المتوقعة.

تحليل وظائف التحقق في: `clsInputValidate`

- سلامة البيانات الشخصية: تقوم `Verify_Email` بالتحقق من الرموز الجوهرية @ و . بينما تفرض `Verify_Phone` طولاً ثابتاً قدره ٩ أرقام فقط.
- المنطق الأكاديمي: تضمن دوال `Verify_cGPA` و `Verify_Semester` بقاء المدخلات ضمن النطاقات المسموحة (٤-٠ للمعدل و ١-٨ للفصل الدراسي).
- التحقق من القسم: تلتزم الدالة `Verify_Department` بالقائمة البيضاء المعتمدة (CS, IT, ME, EE, CE) لرفض أي مدخلات عشوائية.

أدوات الواجهة وتجربة المستخدم: (UX)

يعمل كلاس `clsUI_Utility` على تحسين احترافية التطبيق من خلال ميزة `Pagination` (تقسيم الصفحات)، حيث يتم عرض ١٠ طلاب فقط في كل صفحة (`Page_Size = 10`) هذا يقلل من تشتت البصري ويوفر واجهة تنقل سلسلة (التالي/السابق)، مما يرفع من جودة تجربة المستخدم بشكل كبير مقارنة بالأنظمة التقليدية.

٦. التقييم الفني والتوصيات المستقبلية

يمثل هذا النظام بنية برمجية متماسكة تجمع بين القوة التقنية للغة C++ وسهولة الصيانة التي توفرها هندسة الطبقات. إن الاعتماد على الملفات الرأسية (Header-only) جعل النظام وحدة واحدة سهلة النشر والاستخدام. نقاط القوة الحالية:

١. كفاءة التخزين المباشر: استخدام reinterpret_cast يوفر أداءً لا يضاهى في معالجة الملفات.
٢. التغليف الصارم: حماية الكيانات تضمن عدم وصول قيم غير منطقية إلى طبقة التخزين.
٣. قابلية التوسع: تصميم clsStudentManager يسمح بإضافة معايير فرز أو بحث جديدة دون تعديل الهيكل الأساسي.

التوصيات التقنية المستقبلية:

الأثر المتوقع	التحسين المقترح
تحويل النظام إلى قاعدة بيانات علائقية كاملة لدعم الاستعلامات المتقاطعة المعقدة.	الانتقال إلى SQLite
تطوير واجهة باستخدام Qt لتعزيز سهولة الاستخدام للمستخدم غير التقني.	واجهة رسومية (GUI)
ضبط ترميز النصوص (UTF-8) لدعم إدخال وعرض البيانات باللغة العربية بشكل كامل.	دعم اللغة العربية
إضافة طبقة أمان لإدارة دخول المستخدمين وتحديد صلاحيات (عرض/تعديل/حذف).	نظام صلاحيات (Auth)